

使用 textembedding-gecko@003 取得向量嵌入

來源：<https://codelabs.developers.google.com/codelabs/using-textembedding-gecko?hl=zh-tw>

步驟數：9

在本程式碼研究室中，您將瞭解 gecko@003 模型及應用程式的實際用途。

目錄

1. [1. 簡介](#)
2. [2. 開始設定](#)
3. [3. 匯入必要程式庫](#)
4. [4. 建立模擬向量資料庫](#)
5. [5. 建立文字嵌入](#)
6. [6. 向向量資料庫提問](#)
7. [7. 比較向量](#)
8. [8. 結果](#)
9. [9. 恭喜](#)

1. 簡介

上次更新時間：2024 年 4 月 8 日

文字嵌入

文字嵌入是指將文字資料轉換為數值表示法的程序。這些數字表示法 (通常是向量) 會擷取文字中字詞的語意和關係。想像一下：

文字就像複雜的語言，充滿細微差異和模稜兩可之處。

文字嵌入會將該語言轉換為電腦可解讀及操作的簡化數學格式。

文字嵌入的優點

- 提升處理效率：與原始文字相比，電腦處理數字表示法的速度快得多。這對搜尋引擎、推薦系統和機器翻譯等工作至關重要。
- 擷取語意：嵌入不只會擷取字詞的字面意義，這類模型會擷取字詞之間的脈絡和關係，進行更細緻的分析。
- 提升機器學習效能：文字嵌入可做為機器學習模型中的特徵，進而提升情緒分析、文字分類和主題建模等工作的效能。

文字嵌入的應用實例

文字嵌入技術可將文字轉換為數值表示法，進而解鎖自然語言處理 (NLP) 的各種應用。以下列舉幾個主要用途：

1. 搜尋引擎和資訊檢索：

文字嵌入可讓搜尋引擎瞭解查詢背後的語意，並將查詢與相關文件比對，即使文件中沒有完全相符的關鍵字也沒問題。

搜尋引擎會比較搜尋查詢的嵌入與文件嵌入，找出涵蓋類似主題或概念的文件。

2. 推薦系統：

推薦系統會使用文字嵌入，分析使用者透過評論、評分或瀏覽記錄表達的行為和偏好設定。

接著，系統會比較使用者互動過的產品、文章或其他內容的嵌入，藉此推薦類似項目。

3. 抄襲偵測：

比較兩段文字的嵌入內容，找出語意結構的顯著相似處，有助於找出潛在的抄襲內容。

以上僅列舉幾個例子，隨著文字嵌入技術不斷演進，應用範圍也會持續擴大。隨著電腦透過嵌入內容更瞭解語言，我們預期未來會出現更多創新應用程式。

textembedding-gecko@003

Textembedding-gecko@003 是 Google Cloud Platform (GCP) 透過 Vertex AI 及其 AI 工具和服務套件提供的預先訓練文字嵌入模型特定版本。

建構項目

在本程式碼研究室中，您將建構 Python 指令碼。這個指令碼將：

- 使用 Vertex API 呼叫 textembedding-gecko@003，將文字轉換為文字嵌入 (向量)。
- 建立由文字和向量組成的模擬資料庫
- 比較向量，對模擬向量資料庫執行查詢，並取得最有可能的回覆。

課程內容

- 如何在 GCP 中使用文字嵌入
- 如何呼叫 textembedding-gecko@003
- 如何在 Workbench 中執行這項操作
- 如何使用 Vertex AI Workbench 執行指令碼

軟硬體需求

- 新版 Chrome
 - Python 知識
 - Google Cloud 專案
 - 存取 Vertex AI - Workbench
-

2. 開始設定

建立 Vertex AI Workbench 執行個體

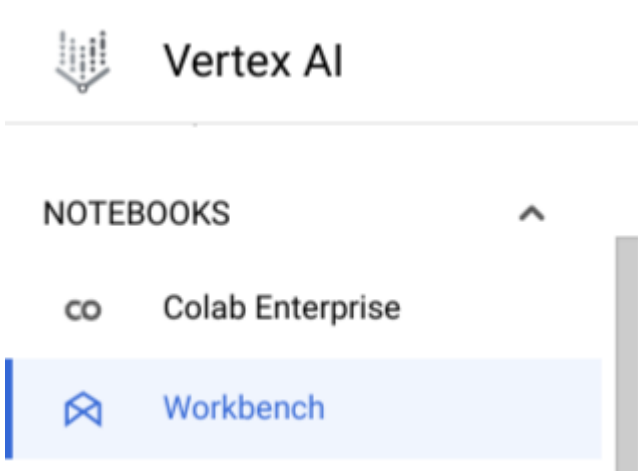
1. 在 Google Cloud 控制台的專案選取器頁面中，選取或建立 Google Cloud 專案。

注意：如果您不打算保留在這項程序中建立的資源，請建立新專案，而不要選取現有專案。這樣在完成這些步驟之後，您就可以刪除專案，並移除與該專案相關聯的所有資源。

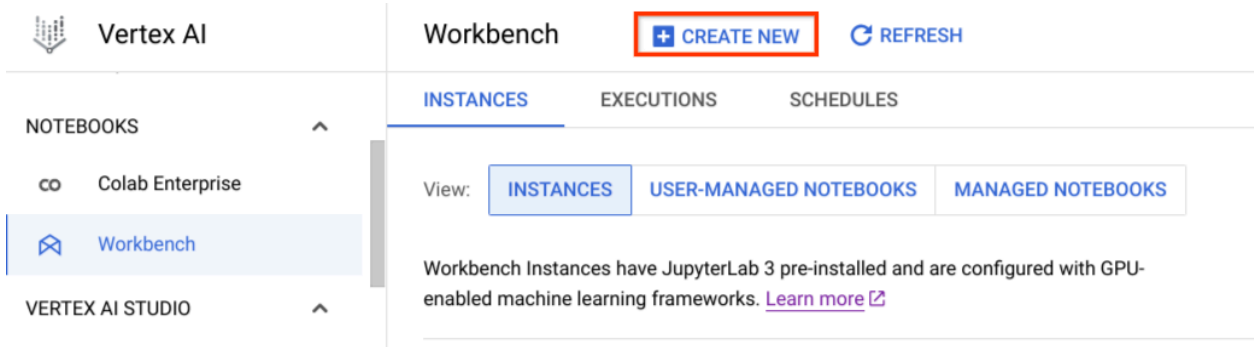
2. 前往專案選取器
3. 請確認 Google Cloud 專案已啟用計費功能。
4. 啟用 Notebooks API。

您可以使用 Google Cloud 控制台、gcloud CLI 或 Terraform 建立 Vertex AI Workbench 執行個體。在本教學課程中，我們將使用 Google Cloud 控制台建立該值區。如要進一步瞭解其他方法，請參閱[這篇文章](#)。

1. 前往 Google Cloud 控制台的「執行個體」頁面 (可透過 Vertex AI 選單的「Notebooks」部分存取)，然後按一下「Workbench」



2. 前往「Instances」(執行個體)。
3. 按一下「建立新項目」



4. 在「建立執行個體」對話方塊的「詳細資料」部分，為新執行個體提供下列資訊：

名稱：為新執行個體命名。名稱開頭必須為英文字母，後面最多可接 62 個小寫英文字母、數字或連字號 (-)，但結尾不得為連字號。

區域和可用區：選取新執行個體的區域和可用區。如要獲得最佳網路效能，請選取最靠近您的地理區域。

無須安裝 GPU

在「網路」部分中，提供下列資訊：

網路：調整網路選項，使用目前專案中的網路，或主專案中的 Shared VPC 網路 (如有設定)。如果您在主專案中使用 Shared VPC，也必須將 Compute 網路使用者角色 (roles/compute.networkUser) 授予服務專案的 Notebooks 服務代理程式。

在「網路」欄位中：選取所需的網路。您可以選取虛擬私有雲網路，只要該網路已啟用 Private Google Access 功能或者可以存取網際網路即可

在「Subnetwork」(子網路) 欄位中：選取所需的子網路。你可以選擇預設的。

在「執行個體屬性」中，您可以保留預設值，也就是 e2-standard-4。

New instance

Name *

instance-20240412-120002

Must start with a letter followed by up to 62 lowercase letters, numbers, or hyphens (-) and cannot end with a hyphen

Region *

us-central1 (Iowa)

▼ ?

Zone *

us-central1-a

▼ ?

☐ Attach 1 NVIDIA T4 GPU

☒ Network in this project

☐ Shared network

Network

default

▼ ?

Subnetwork *

default(10.128.0.0/20)

▼ ?

Instance properties

Machine type	e2-standard-4
Data disk	100 GB Balanced persistent disk
Permission	Compute Engine default service account
Estimated cost ?	\$145.20 monthly, \$0.20 hourly

ADVANCED OPTIONS

CANCEL

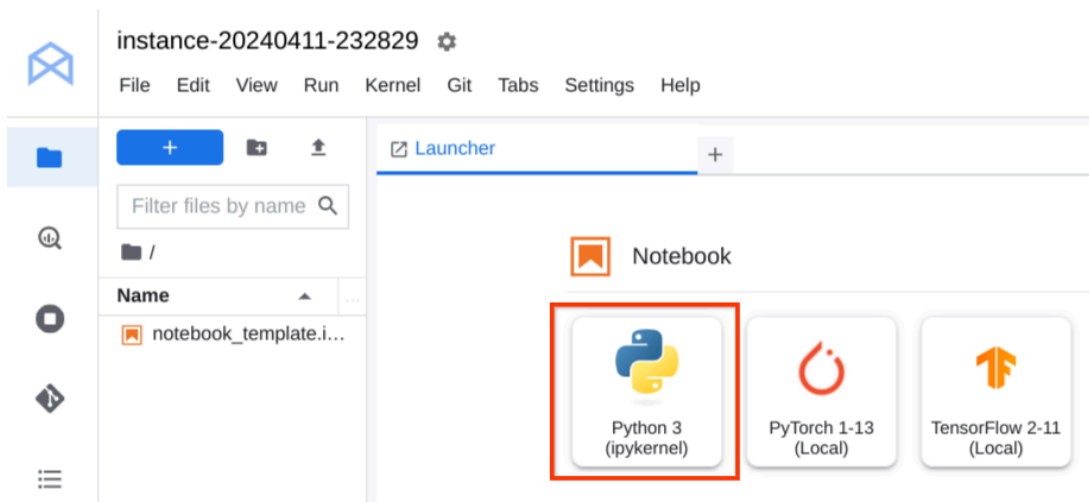
CREATE

5. 按一下「建立」。

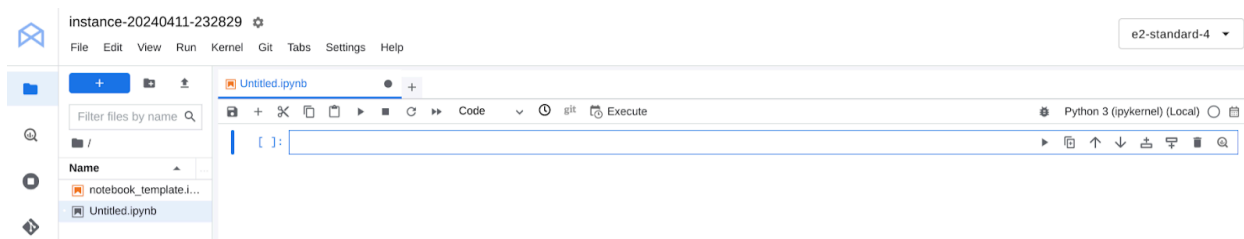
Vertex AI Workbench 會建立執行個體並自動啟動。執行個體可供使用時，Vertex AI Workbench 會啟用「Open JupyterLab」(開啟 JupyterLab) 連結。請點按分頁標籤。

建立 Python 3 筆記本

1. 在 Jupyterlab 中，從啟動器選單的「Notebook」部分，點選標有 Python 標誌的「Python3」圖示。



2. 系統會建立名為「Untitled」的 Jupyter 筆記本，副檔名為 ipynb。



3. 你可以使用左側的檔案瀏覽器部分重新命名，也可以保留原樣。

現在，我們可以開始在筆記本中放入程式碼。

步驟 3 · 約 1 分鐘

3. 匯入必要程式庫

建立執行個體並開啟 Jupyterlab 後，我們需要為程式碼研究室安裝所有必要程式庫。

請準備下列憑證：

1. numpy
2. pandas
3. vertexai.language_models 中的 TextEmbeddingInput、TextEmbeddingModel

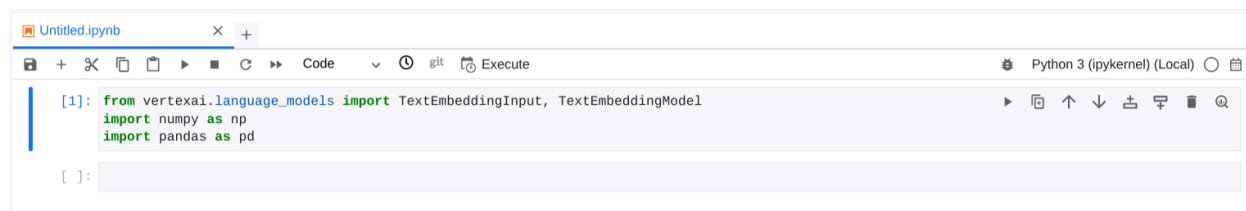
提示：根據預設，Workbench 執行個體已安裝上述所有程式庫，因此我們只需要呼叫這些程式庫即可。

複製下列程式碼並貼到儲存格：

```
from vertexai.language_models import TextEmbeddingInput, TextEmbeddingModel

import numpy as np
import pandas as pd
```

網址看起來會像這樣：



步驟 4 · 約 1 分鐘

4. 建立模擬向量資料庫

為測試程式碼，我們會建立由文字和各自向量組成的資料庫，這些向量是使用 gecko@003 文字嵌入模型翻譯而來。

目標是讓使用者搜尋文字、將文字翻譯成向量、在資料庫中搜尋向量，並傳回最接近的結果。

我們的向量資料庫會保留 3 筆記錄，建立方式如下：

複製下列程式碼，然後貼到新的儲存格。

```
DOCUMENT1 = {
    "title": "Operating the Climate Control System",
    "content": "Your Googlecar has a climate control system that allows you to adjust the temperature and airflow in the car. To operate the climate control system, use the buttons and knobs located on the center console. Temperature: The temperature knob controls the temperature inside the car. Turn the knob clockwise to increase the temperature or counterclockwise to decrease the temperature. Airflow: The airflow knob controls the amount of airflow inside the car. Turn the knob clockwise to increase the airflow or counterclockwise to decrease the airflow. Fan speed: The fan speed knob controls the speed of the fan. Turn the knob clockwise to increase the fan speed or counterclockwise to decrease the fan speed. Mode: The mode button allows you to select the desired mode. The available modes are: Auto: The car will automatically adjust the temperature and airflow to maintain a comfortable level. Cool: The car will blow cool air into the car. Heat: The car will blow warm air into the car. Defrost: The car will blow warm air onto the windshield to defrost it."}

DOCUMENT2 = {
    "title": "Touchscreen",
    "content": "Your Googlecar has a large touchscreen display that provides access to a variety of features, including navigation, entertainment, and climate control. To use the touchscreen display, simply touch the desired icon. For example, you can touch the \"Navigation\" icon to get directions to your destination or touch the \"Music\" icon to play your favorite songs."}

DOCUMENT3 = {
    "title": "Shifting Gears",
    "content": "Your Googlecar has an automatic transmission. To shift gears, simply move the shift lever to the desired position. Park: This position is used when you are parked. The wheels are locked and the car cannot move. Reverse: This position is used to back up. Neutral: This position is used when you are stopped at a light or in traffic. The car is not in gear and will not move unless you press the gas pedal. Drive: This position is used to drive forward. Low: This position is used for driving in snow or other slippery conditions."}

documents = [DOCUMENT1, DOCUMENT2, DOCUMENT3]

df_initial_db = pd.DataFrame(documents)
df_initial_db.columns = ['Title', 'Text']
df_initial_db
```

程式碼應如下所示：

```
[2]: DOCUMENT1 = {
      "title": "Operating the Climate Control System",
      "content": "Your Googlecar has a climate control system that allows you to adjust the temperature and airflow in the car. To operate the c...
      DOCUMENT2 = {
      "title": "Touchscreen",
      "content": "Your Googlecar has a large touchscreen display that provides access to a variety of features, including navigation, entertainm...
      DOCUMENT3 = {
      "title": "Shifting Gears",
      "content": "Your Googlecar has an automatic transmission. To shift gears, simply move the shift lever to the desired position. Park: This...

      documents = [DOCUMENT1, DOCUMENT2, DOCUMENT3]

      df_initial_db = pd.DataFrame(documents)
      df_initial_db.columns = ['Title', 'Text']
      df_initial_db
```

```
[2]:
```

	Title	Text
0	Operating the Climate Control System	Your Googlecar has a climate control system th...
1	Touchscreen	Your Googlecar has a large touchscreen display...
2	Shifting Gears	Your Googlecar has an automatic transmission. ...

分析程式碼

在變數 DOCUMENT1、DOCUMENT2 和 DOCUMENT3 中，我們儲存的字典會模擬文件及其標題和內容。這些「文件」是指 Google 製造的車輛模擬手冊。

在下一行中，我們從這 3 份文件 (字典) 建立清單。

```
documents = [DOCUMENT1, DOCUMENT2, DOCUMENT3]
```

最後，我們運用 pandas 從該清單建立 DataFrame，並將其命名為 df_initial_db。

```
df_initial_db = pd.DataFrame(documents)
df_initial_db.columns = ['Title', 'Text']
df_initial_db
```

步驟 5 · 約 1 分鐘

5. 建立文字嵌入

現在，我們要使用 gecko@003 模型，為模擬文件資料庫中的每筆記錄取得文字嵌入。

複製下列程式碼，然後貼到新的儲存格：

```
def embed_fn(df_input):
    list_embedded_values = []
    for index, row in df_input.iterrows():
        model = TextEmbeddingModel.from_pretrained("textembedding-gecko@003")
        embeddings = model.get_embeddings([(row['Text'])])
        list_embedded_values.append(embeddings[0].values)
    df_input['Embedded text'] = list_embedded_values
    return df_input

df_embedded_values_db = embed_fn(df_initial_db)
df_embedded_values_db
```

程式碼應如下所示：

```
[3]: def embed_fn(df_input):
      list_embedded_values = []
      for index, row in df_input.iterrows():
          model = TextEmbeddingModel.from_pretrained("textembedding-gecko@003")
          embeddings = model.get_embeddings([(row['Text'])])
          list_embedded_values.append(embeddings[0].values)
      df_input['Embedded text'] = list_embedded_values
      return df_input

      df_embedded_values_db = embed_fn(df_initial_db)
      df_embedded_values_db
```

	Title	Text	Embedded text
0	Operating the Climate Control System	Your Googlecar has a climate control system th...	[-0.024551788344979286, -0.016673646867275238, ...
1	Touchscreen	Your Googlecar has a large touchscreen display...	[0.0056630284525454044, -0.02079692855477333, ...
2	Shifting Gears	Your Googlecar has an automatic transmission. ...	[-0.044642843306064606, -0.001312093460010514, ...

分析程式碼

我們定義了名為 `embed_fn` 的函式，該函式會收到包含要執行嵌入作業的文字的 pandas DataFrame 做為輸入內容。然後，函式會傳回編碼為向量的文字。

```
def embed_fn(df_input):
    list_embedded_values = []
    for index, row in df_input.iterrows():
        model = TextEmbeddingModel.from_pretrained("textembedding-gecko@003")
        embeddings = model.get_embeddings([(row['Text'])])
        list_embedded_values.append(embeddings[0].values)
    df_input['Embedded text'] = list_embedded_values
    return df_input
```

我們將在名為 `list_embedded_values` 的清單中，儲存並附加每列的編碼文字。

使用 pandas 的 `iterrows` 方法，我們可以疊代 DataFrame 中的每個資料列，並從「Text」資料欄取得值 (其中包含模擬資料庫的手動資訊)。

如要傳送一般文字並使用 gecko@003 模型傳回向量，請初始化變數模型，並呼叫 `TextEmbeddingModel.from_pretrained` 函式，將模型設為要使用的模型。

```
model = TextEmbeddingModel.from_pretrained("textembedding-gecko@003")
embeddings = model.get_embeddings([(row['Text'])])
```

接著，在變數嵌入中，我們會擷取透過 `model.get_embeddings` 函式傳送的文字向量。

在函式結尾，我們會在資料框架中建立名為「Embedded text」的新資料欄，其中會包含根據 gecko@003 模型建立的向量清單。

```
df_input['Embedded text'] = list_embedded_values
return df_input
```

最後，在 `df_embedded_values_db` 變數中，我們會擷取資料框架，其中包含模擬資料庫的原始資料，以及包含每個資料列向量清單的新資料欄。

```
df_embedded_values_db = embed_fn(df_initial_db)
df_embedded_values_db
```

步驟 6 · 約 1 分鐘

6. 向向量資料庫提問

資料庫現在包含文字和向量，我們可以繼續提問並查詢資料庫，找出答案。

為此，請將下列程式碼複製並貼到新的儲存格：

```
question='How do you shift gears in the Google car?'
model = TextEmbeddingModel.from_pretrained("textembedding-gecko@003")
embeddings = model.get_embeddings([(question)])
text_to_search=embeddings[0].values
len(text_to_search)
```

結果如下所示：

```
[4]: question='How do you shift gears in the Google car?'
      model = TextEmbeddingModel.from_pretrained("textembedding-gecko@003")
      embeddings = model.get_embeddings([(question)])
      text_to_search=embeddings[0].values
      len(text_to_search)

[4]: 768
```

分析程式碼

與上一個步驟的函式類似，我們首先會使用要查詢資料庫的內容初始化問題變數。

```
question='How do you shift gears in the Google car?'
```

接著，我們在模型變數中透過 `TextEmbeddingModel.from_pretrained` 函式設定要使用的模型，在本例中為 `gecko@003` 模型。

```
model = TextEmbeddingModel.from_pretrained("textembedding-gecko@003")
```

在 `embeddings` 變數中，我們會呼叫 `model.get_embeddings` 函式，並傳遞要轉換為向量的文字 (在本例中，我們會傳遞要提出的問題)。

```
embeddings = model.get_embeddings([(question)])
```

最後，`text_to_search` 變數會保存從問題翻譯而來的向量清單。

我們列印向量的長度，僅供參考。

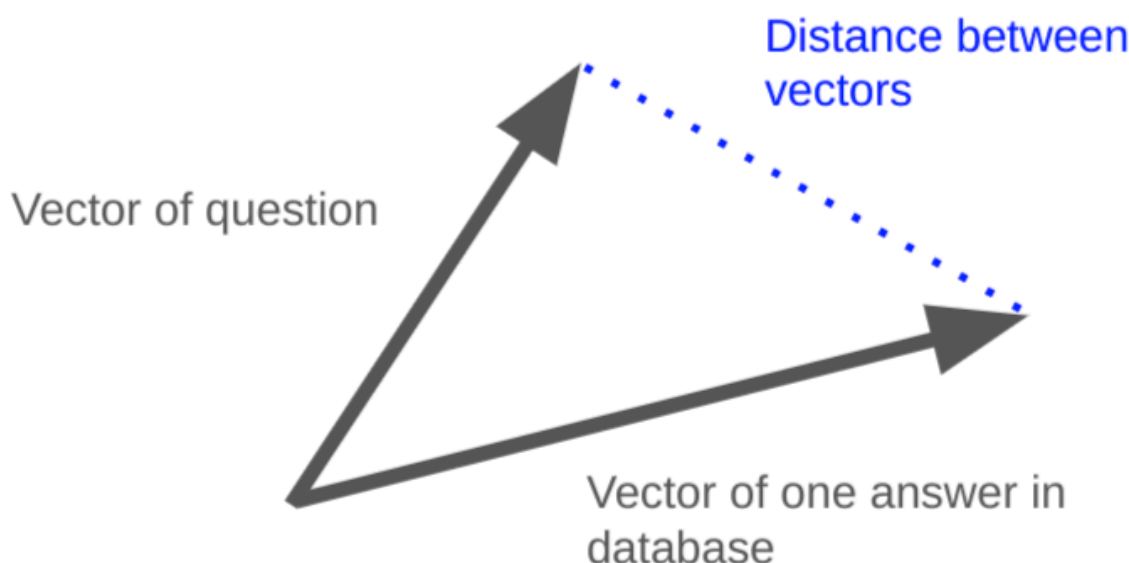
```
text_to_search=embeddings[0].values
len(text_to_search)
```

步驟 7 · 約 1 分鐘

7. 比較向量

現在，我們在模擬資料庫中有一份向量清單，以及一個轉換為向量的問題。也就是說，我們現在可以比較問題的向量與資料庫中的所有向量，找出最接近的向量，更準確地回答問題。

為此，我們會測量問題向量與資料庫中每個向量之間的距離。測量向量間距離的技術有很多種，但本程式碼研究室將使用歐幾里得距離或 L2 範數。



在 Python 中，我們可以運用 NumPy 函式來完成這項工作。

複製下列程式碼，然後貼到新的儲存格：

```
list_embedded_text_from_db = df_embedded_values_db['Embedded text']
shortest_distance=1
for position, embedded_value in enumerate(list_embedded_text_from_db):
    distance=np.linalg.norm((np.array(embedded_value) - np.array(text_to_search)), ord = 2)
    print(distance)
    if distance<shortest_distance:
        shortest_distance=distance
        shortest_position=position

print(f'The shortest distance is {shortest_distance} and the position of that value is {shortest_position}')
```

結果看起來會像這樣：

```
[9]: list_embedded_text_from_db = df_embedded_values_db['Embedded text']
shortest_distance=1
for position, embedded_value in enumerate(list_embedded_text_from_db):
    distance=np.linalg.norm((np.array(embedded_value) - np.array(text_to_search)), ord = 2)
    print(distance)
    if distance<shortest_distance:
        shortest_distance=distance
        shortest_position=position

print(f'The shortest distance is {shortest_distance} and the position of that value is {shortest_position}')
```

0.7406681147172839
0.713414301663597
0.5834923713720351
The shortest distance is 0.5834923713720351 and the position of that value is 2

分析程式碼

首先，我們會將資料庫中包含嵌入文字或向量的資料欄轉換為清單，並儲存在 `list_embedded_text_from_db` 中。

我們也將 `shortest_distance` 變數初始化為 1，以便持續更新，直到找到實際最短距離為止。

```
list_embedded_text_from_db = df_embedded_values_db['Embedded text']
shortest_distance=1
```

接著，我們使用 `for` 迴圈進行疊代，並取得問題向量與資料庫中每個向量之間的距離。

使用 `numpy.linalg.norm` 函式計算距離。

如果計算出的距離小於 `shortest_distance` 變數中的距離，系統就會將計算出的距離設為這個變數

接著，我們會擷取最短距離，以及該距離在清單中的位置。在 `shortest_distance` 和 `shortest_position` 變數中。

```
for position, embedded_value in enumerate(list_embedded_text_from_db):
    distance=np.linalg.norm((np.array(embedded_value) - np.array(text_to_search)), ord = 2)
    print(distance)
    if distance<shortest_distance:
        shortest_distance=distance
        shortest_position=position
```

步驟 8 · 約 1 分鐘

8. 結果

瞭解向量在清單中的位置，即可找出問題與資料庫之間距離最短的向量，然後列印結果。

複製下列程式碼，然後貼到新的儲存格：

```
print("Your question was:\n "+question+ " \nAnd our answer is:\n "+
      df_embedded_values_db.at[shortest_position, 'Title']+"": "+
      df_embedded_values_db.at[shortest_position, 'Text'])
```

執行後，您會看到類似下列的內容：

```
[10]: print("Your question was:\n "+question+ " \nAnd our answer is:\n "+
      df_embedded_values_db.at[shortest_position, 'Title']+"": "+
      df_embedded_values_db.at[shortest_position, 'Text'])
```

Your question was:
How do you shift gears in the Google car?
And our answer is:
Shifting Gears: Your Googlecar has an automatic transmission. To shift gears, simply move the shift lever to the desired position. Park: This position is used when you are parked. The wheels are locked and the car cannot move. Reverse: This position is used to back up. Neutral: This position is used when you are stopped at a light or in traffic. The car is not in gear and will not move unless you press the gas pedal. Drive: This position is used to drive forward. Low: This position is used for driving in snow or other slippery conditions.

步驟 9 · 約 0 分鐘

9. 恭喜

恭喜，您已成功在實際情境中，使用 `textembedding-gecko@003` 模型建構第一個應用程式！

您已瞭解文字嵌入的基礎知識，以及如何在 GCP Workbench 上使用 `gecko003` 模型。

您現已瞭解，如要將所學知識應用於更多用途，必須採取哪些重要步驟。

後續步驟

查看一些程式碼研究室...

- [開始透過 AlloyDB AI 使用向量嵌入](#)

參考文件

- [文字嵌入](#)
 - [Vertex AI Workbench 簡介](#)
 - [使用嵌入項目搜尋文件](#)
-